Jean Victor Borges de Abreu

# Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Petrópolis - Rio de Janeiro
2026

Jean Victor Borges de Abreu

# Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Trabalho de Conclusão de Curso apresentado à...

FACULDADE DE EDUCAÇÃO TECNOLÓGICA DO ESTADO DO RIO DE JANEIRO
FAETERJ/PETRÓPOLIS

Orientador: Nome do Orientador

Petrópolis - Rio de Janeiro
2026

# 1 Resumo

# 2 Abstract

# 3 Lista de Figuras

# 4 Lista de Abreviaturas

FAETERJ: Faculdade de Educação Tecnológica do Estado do Rio de Janeiro
TRNG: True Random Number Generator

# Sumário


1 RESUMO . . . . . 2

2 ABSTRACT . . . . . 3

3 LISTA DE FIGURAS . . . . . 4

4 LISTA DE ABREVIATURAS . . . . . 5

5 INTRODUÇÃO . . . . . 7
5.1 Contextualização do Tema . . . . . 7
5.2 Justificativa/Problema de Pesquisa . . . . . 7
5.3 Motivação/Hipotese e Ameaca Tecnológica . . . . . 8
5.4 Objetivos . . . . . 8
5.5 Metodologia . . . . . 8

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO . 9
6.1 Aleatoriedade Verdadeira vs. Pseudoaleatoriedade . . . . . 9
6.2 Entropia e Fontes de Incerteza . . . . . 9
6.3 A Semente (Seed) como Elemento Crítico . . . . . 10
6.4 Avaliação Estatística da Aleatoriedade . . . . . 10

7 GERADORES CRIPTOGRAFICAMENTE SEGUROS . . . . . 11
7.1 PRNGs vs. CSPRNGs . . . . . 11
7.2 O Ataque do Próximo Bit . . . . . 11
7.2.1 Implementação Prática (Demonstração) . . . . . 11
7.3 Implementação de CSPRNGs em Java . . . . . 11
7.4 Falhas Históricas de Aleatoriedade . . . . . 11

8 COMPUTAÇÃO QUÂNTICA E A QUEBRA DO PARADIGMA ATUAL . . . . . 12
8.1 Conceitos Fundamentais de Computação Quântica . . . . . 12

8.2 Algoritmo de Shor . . . . . 12
8.3 Algoritmo de Grover . . . . . 12
8.4 Harvest Now, Decrypt Later . . . . . 12

9 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC) . . . . . 13
9.1 Definição e Objetivos da PQC . . . . . 13
9.2 Paradigmas Matemáticos da PQC . . . . . 13
9.3 Criptografia Baseada em Reticulados . . . . . 13
9.4 Learning With Errors (LWE) . . . . . 13
9.4.1 Implementação Didática em Java (Demonstração) . . . . . 13
9.5 Padronização Pós-Quântica . . . . . 13

10 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA . . . . . 14
10.1 Suporte à Criptografia Pós-Quântica no Java . . . . . 14
10.2 Crypto-Agility . . . . . 14
10.2.1 Implementação Prática: Demo de Crypto-Agility . . . . . 14
10.3 Comparativo de Modelos . . . . . 14
10.3.1 Implementação Prática: Benchmark Clássico vs. Kyber . . . . . 14

11 DISCUSSÃO E ANÁLISE CRÍTICA . . . . . 15
11.1 Impacto em Desempenho . . . . . 15
11.2 Barreiras de Adoção . . . . . 15
11.3 Implicações Sociais e Éticas . . . . . 15

12 DISCUSSÃO E ANÁLISE CRÍTICA . . . . . 16
12.1 Síntese dos Resultados . . . . . 16
12.2 Resposta ao Problema de Pesquisa . . . . . 16
12.3 Trabalhos Futuros . . . . . 16

13 BIBLIOGRAFIAS UTEIS . . . . . 17

14 COISAS POSSIVELMENTE UTEIS EM ALGUM MOMENTO . . 19

15 CONCLUSÃO . . . . . 21

16 EXEMPLOS PARA USO DA BIBLIOGRÁFIA . . . . . 22

REFERÊNCIAS . . . . . 23

# 5 Introdução

A onipresença da criptografia na infraestrutura digital moderna (internet, sistemas bancários, IoT e comunicações seguras).

A infraestrutura de segurança digital de grandes corporações, em sua maioria sustentada por ecossistemas como o Java, vivencia um momento de transição crítica. Visto que computadores operam sob lógica determinística e não podem gerar o 'acaso puro' sem fontes externas de entropia, a criptografia contemporânea torna-se dependente da robustez matemática dos Geradores de Números Pseudo Aleatórios (PRNGs).

Durante décadas, bibliotecas de segurança confiaram em implementações consolidadas, como a classe java.security.SecureRandom, para garantir a integridade de chaves e certificados. Contudo, o setor presencia o ocaso da criptografia clássica: a padronização recente liderada pelo NIST indica que as fontes de entropia e os algoritmos de pseudo aleatoriedade tradicionais não são mais suficientes contra ataques baseados em vetores quânticos. Nesse cenário, se o gerador de números responsável pela criação de chaves for previsível para um sistema quântico, a estrutura de segurança da informação como um todo torna-se obsoleta.

O presente artigo explora esse panorama de mudança, avaliando como a transição para a Criptografia Pós-Quântica (PQC) altera as práticas de desenvolvimento e investigando novos padrões de redes de pontos (lattices), bem como a adaptação da plataforma Java para assegurar a soberania dos dados em um futuro pós-quântico.

## 5.1 Contextualização do Tema

Com o avanço da tecnologia e o uso crescente de ferramentas e hardwares quanticos. A utilização de meios pesudoaleatorios tem sidos colocados em risco. E sua utilização em alguns anos podem se tornar obsoletos e inseguros para utilização. Como já é o caso de grandes empresas e organizações governamentais. Das quais já fazem uso de outros meios para criptografar suas informações.

## 5.2 Justificativa/Problema de Pesquisa

O conflito entre o determinismo computacional e a necessidade de aleatoriedade em sistemas criptográficos.

## 5.3 Motivação/Hipotese e Ameaca Tecnológica

O impacto da computação quântica — especialmente por meio do Algoritmo de Shor — sobre os modelos atuais de segurança e pseudoaleatoriedade.

## 5.4 Objetivos

Objetivo Geral: Analisar a evolução dos geradores pseudoaleatórios criptograficamente seguros frente à transição para a criptografia pós-quântica. Objetivos Específicos: Estudar PRNGs e CSPRNGs clássicos Analisar o impacto dos algoritmos quânticos Investigar os novos paradigmas de pseudoaleatoriedade baseados em reticulados Avaliar a adoção desses conceitos no ecossistema Java

## 5.5 Metodologia

Pesquisa bibliográfica, análise documental de padrões criptográficos e estudo prático no ecossistema Java.

# 6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO

## 6.1 Aleatoriedade Verdadeira vs. Pseudoaleatoriedade

Definições de e PRNG (Pseudo-Random Number Generator).

TRNG é um dispositivo que gera números aleatórios a partir de um processo físico, e não por meio de um algoritmo. Tais dispositivos são frequentemente baseados em fenômenos microscópicos que geram sinais de "ruído"estatisticamente aleatórios de baixo nível, como ruído térmico, efeito fotoelétrico, ou fenômenos quânticos. Esses processos estocásticos são, em teoria, completamente imprevisíveis enquanto uma equação que governa tais fenômenos for desconhecida ou incomputável. Isso contrasta com o paradigma de geração de números pseudo- aleatórios comumente implementado através de algoritmos.(ALVES, 2022)

PRNG refere-se a uma sequência de números ou dados que aparenta ser aleatória, mas é gerada por um algoritmo determinístico. E é gerada por algoritmos que utilizam fórmulas matemáticas para produzir resultados aparentemente aleatórios. Embora sequências pseudoaleatórias possam imitar a aleatoriedade até certo ponto, elas são, em última análise, determinísticas e repetíveis. Esses algoritmos recebem um valor inicial, chamado semente, e o utilizam para produzir uma sequência de números. Ao aplicar operações matemáticas ao valor da semente, o PRNG produz uma série de números aparentemente aleatórios. O valor da semente pode ser escolhido arbitrariamente, mas afeta significativamente a sequência resultante de números pseudoaleatórios.(Lenovo, s.d.)

## 6.2 Entropia e Fontes de Incerteza

Captação de eventos físicos imprevisíveis como base para sistemas criptográficos seguros.

Entropia é uma medida da aleatoriedade ou imprevisibilidade dos dados. No contexto da criptografia, a entropia é usada para gerar números aleatórios ou chaves que são essenciais para a comunicação segura e a criptografia. Sem uma boa fonte de entropia, os protocolos criptográficos podem se tornar vulneráveis a ataques que exploram a previsibilidade das chaves geradas.

Na maioria dos sistemas operacionais, a entropia é gerada pela coleta de eventos aleatórios de diversas fontes, como interrupções de hardware, movimentos do mouse, pressionamentos de teclas e atividade do disco. Esses eventos são inseridos em um conjunto de entropia, que é então usado para gerar números aleatórios quando necessário.

Protocolos criptográficos, como SSL/TLS, SSH e IPsec, dependem fortemente da entropia para gerar números aleatórios e chaves para criptografia, autenticação e proteção

de integridade. A aleatoriedade desses números é crucial para a segurança da comunicação. Se um atacante conseguir prever os números aleatórios usados em um protocolo criptográfico, ele poderá comprometer a confidencialidade, a integridade ou a autenticidade da comunicação. Se a chave mestra prévia for previsível ou tendenciosa, um atacante poderá interceptar a mensagem criptografada, descriptografá-la e obter acesso às chaves de sessão. Isso permitiria ao atacante espionar a comunicação, modificar os dados ou se passar pelo servidor.(NETDATA, s.d.)

## 6.3 A Semente (Seed) como Elemento Crítico

O papel do valor inicial na previsibilidade e segurança de sequências pseudoaleatórias.

## 6.4 Avaliação Estatística da Aleatoriedade

Bateria de testes SP 800-22 do NIST e o conceito de indistinguibilidade estatística.

# 7 Geradores Criptograficamente Seguros

## 7.1 PRNGs vs. CSPRNGs

Diferenças arquiteturais e requisitos de segurança.

## 7.2 O Ataque do Próximo Bit

Por que geradores comuns falham em contextos criptográficos.

### 7.2.1 Implementação Prática (Demonstração)

Demonstração do ataque contra java.util.Random, mostrando previsão de saídas futuras com base em observações anteriores — contrasta diretamente com o CSPRNG.

## 7.3 Implementação de CSPRNGs em Java

Análise da classe java.security.SecureRandom e seus provedores (SHA1PRNG, NativePRNG).

## 7.4 Falhas Históricas de Aleatoriedade

Casos emblemáticos como o Debian OpenSSL e problemas recorrentes em dispositivos IoT.

# 8 COMPUTAÇÃO QUÂNTICA E A QUEBRA DO PARADIGMA ATUAL

## 8.1 Conceitos Fundamentais de Computação Quântica

Diferença entre bits clássicos e qubits.

## 8.2 Algoritmo de Shor

Fatoração eficiente e o colapso dos sistemas baseados em RSA e ECC.

## 8.3 Algoritmo de Grover

Impacto sobre criptografia simétrica e necessidade de aumento no tamanho das chaves.

## 8.4 Harvest Now, Decrypt Later

A urgência da migração para algoritmos resistentes a ataques quânticos.

# 9 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC)

## 9.1 Definição e Objetivos da PQC

Algoritmos clássicos projetados para resistir a ataques quânticos.

## 9.2 Paradigmas Matemáticos da PQC

Reticulados, códigos corretores de erro, funções hash e isogenias.

## 9.3 Criptografia Baseada em Reticulados

Introdução ao conceito de Lattices como base da nova segurança criptográfica.

## 9.4 Learning With Errors (LWE)

Fundamento matemático da pseudoaleatoriedade pós-quântica.

### 9.4.1 Implementação Didática em Java (Demonstração)

Implementação simplificada do LWE do zero em Java para fins didáticos, mostrando como o erro intencional funciona matematicamente antes de utilizar a biblioteca pronta.

## 9.5 Padronização Pós-Quântica

O papel do ML-KEM (Kyber) e o processo de padronização conduzido pelo NIST.

# 10 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA

## 10.1 Suporte à Criptografia Pós-Quântica no Java

Evolução do JDK e integração de algoritmos pós-quânticos em versões modernas (Java 21+).

## 10.2 Crypto-Agility

Boas práticas para desenvolvimento de sistemas criptográficos flexíveis e preparados para mudanças de algoritmo.

### 10.2.1 Implementação Prática: Demo de Crypto-Agility

Criação de uma interface Java que abstrai o algoritmo de geração de chaves, permitindo trocar de RSA para Kyber alterando apenas a configuração — sem reescrever o código da aplicação.

## 10.3 Comparativo de Modelos

Geração de chaves e aleatoriedade no modelo clássico versus modelo baseado em reticulados (Module-Lattice).

### 10.3.1 Implementação Prática: Benchmark Clássico vs. Kyber

Benchmark comparativo entre java.security.SecureRandom clássico e ML-KEM (Kyber) via Bouncy Castle, medindo tempo de geração de chaves, uso de memória e throughput. Inclui comparativo de tamanho de chaves entre RSA-2048 e Kyber. Nota: Confirmar com o orientador o uso da biblioteca Bouncy Castle como dependência, por ser a opção mais acessível para PQC em Java atualmente.

# 11 DISCUSSÃO E ANÁLISE CRÍTICA

## 11.1 Impacto em Desempenho

Custos computacionais, consumo de memória e latência — baseados nos resultados do benchmark da seção 6.3.1.

## 11.2 Barreiras de Adoção

Sistemas legados, limitações de hardware e resistência organizacional.

## 11.3 Implicações Sociais e Éticas

Soberania de dados, privacidade, vigilância e assimetria tecnológica.

# 12 DISCUSSÃO E ANÁLISE CRÍTICA

## 12.1 Síntese dos Resultados

A inevitabilidade da transição criptográfica.

## 12.2 Resposta ao Problema de Pesquisa

A evolução da pseudoaleatoriedade como pilar da segurança pós-quântica.

## 12.3 Trabalhos Futuros

Geradores Quânticos de Números Aleatórios (QRNG) e integração híbrida TRNG + PQC.

# 13 Bibliografias Uteis

1. Fundamentos e Estado da Arte (Capítulos 2 e 3)

Para falar de aleatoriedade, sementes e o funcionamento dos CSPRNGs atuais:

Livro: The Art of Computer Programming, Vol 2: Seminumerical Algorithms (Donald Knuth).

Onde usar: Capítulos 2.1 e 3.1. É a base matemática para entender geradores lineares e por que eles são previsíveis.

Livro: Handbook of Applied Cryptography (Menezes, van Oorschot e Vanstone).

Onde usar: Capítulo 3.2 e 4.1. Consulte o Capítulo 5 (Pseudorandom Bits and Sequences). Ele define formalmente o "Next Bit Test".

Norma Técnica: NIST SP 800-22 Rev 1a: A Statistical Test Suite for Random and Pseudorandom Number Generators.

Onde usar: Capítulo 2.4. É o documento que dita como o mundo testa se um PRNG é bom.

2. A Ameaça Quântica (Capítulo 4)

Aqui você precisa citar os fundamentos da física quântica aplicados à quebra de códigos:

Livro: Quantum Computation and Quantum Information (Michael Nielsen e Isaac Chuang).

Onde usar: Capítulo 4.1 e 4.2. É a "bíblia"da computação quântica. Essencial para explicar Qubits e o Algoritmo de Shor.

Artigo Seminal: Algorithms for quantum computation: discrete logarithms and factoring (Peter Shor, 1994).

Onde usar: Capítulo 4.2. O artigo que mudou a história da criptografia.

Relatório: NIST IR 8105: Report on Post-Quantum Cryptography.

Onde usar: Capítulo 4.4. Explica o conceito de "Harvest Now, Decrypt Later".

3. Criptografia Pós-Quântica e Reticulados (Capítulo 5)

Esta é a parte mais densa e moderna do seu trabalho:

Artigo de Revisão: A Decade of Lattice Cryptography (Chris Peikert).

Onde usar: Capítulo 5.3 e 5.4. Peikert é um dos maiores nomes da área. Este artigo resume como a matemática de reticulados (Lattices) se tornou segura.

Artigo Técnico: The Learning with Errors Problem (Oded Regev).

Onde usar: Capítulo 5.4. Oded Regev criou o conceito de LWE. Este artigo é a fonte primária para a sua fundamentação matemática.

Documento Oficial (FIPS): FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard.

Onde usar: Capítulo 5.5. Este é o padrão final do ML-KEM (Kyber). É a referência mais importante para o seu futuro capítulo 6.

4. Transição e Ecossistema Java (Capítulo 6)

Para a parte teórica sobre Java e Agilidade Criptográfica:

Documentação JEP (JDK Enhancement Proposals):

JEP 466 (Class-File API) e documentos relacionados à implementação do ML-KEM no provedor SunJCE.

Onde usar: Capítulo 6.1.

Artigo sobre Crypto-Agility: A Taxonomy of Crypto-Agility (S. Ott et al.).

Onde usar: Capítulo 6.2. Ajuda a definir por que sistemas modernos não podem ficar "presos"a um único algoritmo.

# 14 Coisas possivelmente Uteis em algum momento

PRNG é uma sigla para “Pseudo Random Number Generator” ou “Gerador de Números Pseudo Aleatórios”. Seu significado é que a sua geração de números não segue uma estrutura aleatória. Ele utiliza um ponto inicial denominado “seed” para dar início a sua geração de números. A seed é um valor base para ser utilizado no PRNG. Utilizando a mesma seed, o resultado final do PRNG será o mesmo.

O PRNG funciona a partir de uma função que recebe a seed por parâmetro. E através de um cálculo, ele retorna o resultado dessa fórmula. Uma das fórmulas mais comuns para esse tipo de gerador de pseudo números aleatórios é no modelo LCG “Gerador Congruencial Linear”

$$\boldsymbol{x_{i+1} = (A \cdot x_i + C) \pmod{M}} \tag{14.1}$$

Onde o valor inicial “x0” é a seed, A e C são parâmetros inteiros que podem ter um valor já pré-definido ou passados diretamente, M é o espaço dos valores aleatórios possíveis e “mod” é o restante da divisão inteira.

Figura 1 – Modelo simples de um PRNG com LCG

```java
public class Main {
    public static void main(String[] args) {
        long seed = 12345;
        long a = 1664525;
        long c = 1013904223;
        long mod = (long) Math.pow(2, 32);

        LCG lcg = new LCG(seed, a, c, mod);

        for (int i = 0; i < 10; i++) {
            System.out.println(lcg.next());
        }

    }

    public static class LCG {  2 usages

        private long a;  2 usages
        private long c;  2 usages
        private long mod;  2 usages
        private long xi;  4 usages

        public LCG(long seed, long a, long c, long mod) {  1 usage
            this.xi = seed;
            this.a = a;
            this.c = c;
            this.mod = mod;
        }

        public long next() {  1 usage
            xi = (a * xi + c) % mod;
            return xi;
        }
    }
}
```

Fonte: Elaborado pelo autor (2026)

Mesmo após inúmeras tentativas. Devido a seed permanecer inalterada. O resultado permanece o mesmo em todos os resultados obtidos.

Por mais que tenha se passado um valor fixo na seed no exemplo prático, a seed é definida pelo sistema operacional, esse número é denominado entropia. Que se tem por referência a quantidade de imprecisão de um sistema. Quanto maior o número da entropia, maior é a incerteza. Esse valor de entropia é gerado a partir de certos eventos que ocorrem no momento de sua captura, como por exemplo o movimento do mouse, os inputs no teclado etc. Essas capturas são convertidas em um valor numérico para ser usado na seed.

Esses inputs que o sistema captura podem ser previsíveis, tornando a entropia causada menor. Sendo possível pré-determinar o valor gerado ao final para seed. Apesar das aplicações para o dia a dia ou em pequenos projetos serem o suficiente. Esse método não é seguro para aplicações de alto risco.

Figura 2 – Resultado do script LCG

```
/app/extra/jbr/bin/java -javaagent:/app/extra/lib/idea_rt.jar=34643 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encodin
87628868
71072467
2332836374
2726892157
3908547000
483019191
2129828778
2355140353
2560230508
3364893915

Process finished with exit code 0
```

Fonte: Elaborado pelo autor (2026)

# 15 Conclusão

Considerações finais.

# 16 Exemplos para uso da bibliográfia

para livros (KNUTH, 1997)

para artigos (MATSUMOTO; NISHIMURA, 1998)

para sites e outras fontes (Oracle Corporation, 2023)

para outros TCC e afins (ALMEIDA, 2025)

para fichas tecnicas e documentacao (International Organization for Standardization, 2022)

# Referências

ALMEIDA, C. *Análise de Geradores de Números Aleatórios em Sistemas Embarcados*. Dissertação (Dissertação (Mestrado em Engenharia Eletrônica)) — Universidade Estadual do Rio de Janeiro, Rio de Janeiro, 2025.

ALVES, P. *Desenvolvimento de TRNG (True Random Number Generator) através de células memristoras*. Dissertação (Trabalho de Conclusão de Gradução) — Universidade Federal do Rio Grande do Sul, Porto Alegre, 2022. Disponível em: <https://lume.ufrgs.br/handle/10183/251751>. Acesso em: 18 fev. 2026.

International Organization for Standardization. *ISO/IEC 27001: Information technology - Security techniques*. Geneva, CH, 2022.

KNUTH, D. E. *The Art of Computer Programming: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 1997. v. 2.

Lenovo. *Glossary: Pseudorandom*. s.d. Disponível em: <https://www.lenovo.com/us/en/glossary/pseudorandom>. Acesso em: 18 fev. 2026.

MATSUMOTO, M.; NISHIMURA, T. Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, v. 8, n. 1, p. 3–30, 1998.

NETDATA. *Understanding Entropy Key To Secure Cryptography & Randomness*. s.d. Disponível em: <https://www.netdata.cloud/blog/understanding-entropy-the-key-to-secure-cryptography-and-randomness>. Acesso em: 18 fev. 2026.

Oracle Corporation. *Class Random: Java Platform SE 17*. 2023. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html>. Acesso em: 18 fev. 2026.